

Week 1: Car Rental Planning

CIS 453 — Car Rental Service

Weekly Report: Week 1 — Planning & Requirements Gathering

Team Name / Members: JJKC — Joshua Ane, Jae Nicasio, Chisimdi Ikeanus, Kwadwo Adubofour

1. Objectives for this Week

1. Determine and document whether the Booking platform will support only dealership-listed vehicles for rental or also permit customers to list their own vehicles for rental. Clearly articulate the rationale for the chosen approach to ensure alignment with project requirements and facilitate transparent decision-making.
 2. Write out the functional requirements, covering the five things the assignment asks for: registration/login, browsing by category, booking for a date range, payment and confirmation, and admin management.
 3. Get a first draft of the SRS done.
 4. Create a high-level architecture diagram showing the system's main components and how they connect.
 5. Set up the GitHub repo and a shared doc, and split up who writes what so we don't overlap.
-

2. Work Completed

Scope and business model defined. We are building a rental platform for a single local car dealership that owns its own fleet. The dealership is the only administrator. Customers can register, browse, and book, but cannot list their own vehicles.

We considered a peer-to-peer marketplace model, where any user could both rent and list cars, and rejected it. The scope decision below explains why a marketplace specifically doesn't fit; the two points that follow explain why moving to an online booking system at all (marketplace or not) benefits both the customer and the dealership:

- **Administration stays simple.** The system assumes a single admin managing a single fleet under one set of rental policies, rather than multiple independent owners. A marketplace would require per-owner accounts and permissions, owner verification, payout splitting, and dispute handling between two customer classes. That is a lot of design work for a four-week prototype, and it would solve a problem this dealership doesn't have: there's only one owner and one customer class, so there's nothing to split payouts between and no second party to dispute with.
- **The system saves the customer a trip.** Under the current process, a customer must come to the dealership to check availability and sign the agreement. Browsing availability, selecting dates, and confirming a booking online moves all of that to the customer's own time, before they ever drive over.
- **The system reduces dealership costs.** Digitising the rental agreement and availability calendar eliminates the paper file and manual, whiteboard-style tracking, reducing staff hours spent on rental paperwork and phone availability checks.

Drafted the functional requirements as a key deliverable for this week. These are organised into sixteen numbered specifications, grouped by the relevant system actors.

ID	Requirement
FR-1	A visitor can register an account with name, email, phone number, and password.
FR-2	A registered customer can log in and log out; sessions are authenticated.
FR-3	A customer can browse the fleet and filter by category (economy, sedan, SUV, truck).
FR-4	A customer can view a vehicle detail page with the make, model, year, category, daily rates, and photos.
FR-5	A customer can enter a pickup and return date and see only the vehicles available for that full range.

FR-6	A customer can create a booking for an available vehicle for a selected date range; the system calculates the total cost from the daily rate × number of days.
FR-7	A customer can complete a payment for their booking.
FR-8	Upon successful payment, the customer sees an on-screen confirmation showing the booking reference.
FR-9	Upon successful payment, the system sends the customer an email containing the booking reference
FR-10	A customer can view their booking history and cancel a booking that has not started.
FR-11	An admin can add, edit, and remove vehicles in the fleet.
FR-12	An admin can change a vehicle's status between available, rented, and out of service.
FR-13	An admin can block a date range on a vehicle's calendar for maintenance.
FR-14	An admin can view all bookings and filter them by date or status.
FR-15	An admin can mark a vehicle as returned, closing out its associated booking.
FR-16	The system shall reject a booking attempt that overlaps an existing confirmed booking for the same vehicle.

Non-functional requirements

- Passwords are stored hashed, never in plain text.
- Role-based access control: customer routes and admin routes are separated.

SRS drafted. Sections written: introduction and purpose, scope and out-of-scope list, actor definitions (visitor, customer, admin), the functional requirements above, non-functional requirements, a stated assumptions section, and supporting diagrams to illustrate key architectural components.

High-level architecture diagram produced. A three-tier block diagram:

- **Presentation layer** — browser-based UI with two route groups, customer-facing and admin-facing.
- **Application layer** — five logical modules: Authentication, Vehicle Catalogue, Booking, Payment, and Admin Management. The Booking module owns availability validation because that is where the overlap rule must be enforced.
- **Data layer** — a relational database with four core tables: Users, Vehicles, Bookings, and Payments.

Out of scope, recorded explicitly: multi-location or multi-dealership support, customer-listed vehicles, real payment processing, insurance add-on products, loyalty programs, delivery and pickup scheduling, and damage or incident reporting.

3. Challenges Encountered

Deciding where availability lives in the data model. We debated whether each vehicle should maintain its own availability calendar, or whether availability should be derived by querying existing bookings for date overlaps. This matters because it determines the schema and which module prevents double-booking. We are leaning toward deriving availability from the Bookings table, since a separate calendar creates two sources of truth that can drift apart, but haven't finalised it. As a result, the data model isn't locked, which pushes some Week 1 design work into Week 2.

Payment cannot be implemented in real time. We can't accept real card payments in a student project, and we shouldn't handle real card data at all. This left us unsure how to write FR-7 without either overpromising or gutting a requirement the assignment explicitly asks for. We resolved it by scoping payment as a simulated transaction: the system records a paycord, generates a confirmation, and marks the booking as confirmed without any external gateway. We've now added this to the SRS assumptions section so our design isn't later evaluated against a real integration we never claimed.

Scope creep during brainstorming. Early discussion pulled in insurance add-ons, a loyalty program, delivery scheduling, and damage reporting. Each sounded reasonable in isolation, and none of them is in the assignment's minimum functionality. We lost most of one meeting to this before agreeing to keep a written out-of-scope list and treat it as closed unless the whole team reopens it.

Coordination and duplicate work. Before we split ownership, two members independently started writing overlapping requirements documents, and reconciling them cost time we didn't need to spend. Scheduling was also tight given differing class blocks. We moved to a single shared document with named section owners and a standing meeting time, which should prevent a repeat.

4. Next Steps (Week 2 — System Architecture & Design)

1. **Lock the data model.** Settle the availability question, then finalise the Users, Vehicles, Bookings, and Payments tables with fields, types, keys, and the constraint that prevents overlapping bookings per vehicle. Owner: Chisimdi.
2. **Map requirements to design.** Build a traceability table with one row per functional requirement, naming the component and class responsible, so nothing in the SRS is left unassigned. Owner: Jae.
3. **Produce the class diagram.** Classes for User, Customer, Admin, Vehicle, Booking, and Payment, with attributes, methods, and relationships. Owner: Joshua . Tool: [Visual Paradigm].
4. **Produce dynamic flow diagrams** for the two flows most likely to break: the booking flow from date selection through confirmation, and the admin flow for taking a vehicle out of service while a future booking exists against it. Owner: Kwadwo.
5. **Choose and justify the technology stack** (Phase 4). Propose and compare two candidate technology stacks on the same criteria — team familiarity, speed of getting a working prototype in four weeks, and database fit. Write one paragraph of justification per layer: language, framework, database, and tooling. Owner: whole team, decided on 09/16/2026.
6. **Add repository structure:** README, a folder structure separate from /src, and a branch convention, so week 3 implementation can start without delays. Owner: Joshua.

5. Team Contribution

Member	Tasks Completed This Week
Kwadwo	Facilitated the scope decision and wrote the business-model rationale; drafted the SRS introduction, scope, and out-of-scope sections.
Jae	Wrote the customer-facing functional requirements (FR-1 through FR-10) and the actor definitions
Chisimdi	Wrote the admin functional requirements (FR-11 through FR-16) and the non-functional requirements; researched the overlapping-booking constraint.
Joshua	Produced the high-level architecture block diagram; created the GitHub repository and shared document workspace.

All four members participated in the scope debate and signed off on the out-of-scope list.